🟢 SESIÓN 7 Service Layer + @Transactional + Lógica de negocio

"El controller no piensa."

🟦 Bloque 1 — Problema real

En sesión 6 el controller ya hace demasiado:

- resuelve categorías
- decide si crea categoría o la reutiliza
- guarda libro
- mapea a DTO
- decide status

👉 Eso mezcla responsabilidades y complica el mantenimiento.

💡 Mensaje clave:

"El controller orquesta HTTP. El service resuelve el negocio."

🟦 Bloque 2 — Teoría esencial

¿Qué aporta el Service Layer?

- Centraliza la lógica
- Reutilización (un mismo caso de uso desde varios endpoints)
- Controllers más simples
- Punto natural para reglas de negocio (ej: "no duplicar categoría", "no permitir updates si...")

¿Qué aporta @Transactional?

Asegura consistencia cuando una operación implica varias acciones:

Ejemplo create book:

1. encontrar/crear categoría
2. guardar book

Si algo falla a mitad → **rollback** (todo o nada).

💡 **Mensaje clave:** "Una operación de negocio debe ser atómica."

🟦 Bloque 3 — Estructura del proyecto

```
com.cladsb1.books
 ├── controller
 │   └── BookController
 ├── service
 │   └── BookService
 ├── entity
 │   ├── Book
 │   └── Category
 ├── repository
 │   ├── BookRepository
 │   └── CategoryRepository
 ├── dto
 │   ├── request
 │   │   └── BookRequestDTO
 │   └── response
 │   └── BookResponseDTO
 ├── error
 │   ├── ApiError
 │   └── GlobalExceptionHandler
 └── BooksApplication
```

✅ A) VERSIÓN CON STREAMS / LAMBDAS

🟦 Bloque 4A — BookService (CON streams/lambdas)

📁 **service/BookService.java**

```
package com.cladsb1.books.service;

import com.cladsb1.books.dto.request.BookRequestDTO;
import com.cladsb1.books.dto.response.BookResponseDTO;
import com.cladsb1.books.entity.Book;
import com.cladsb1.books.entity.Category;
import com.cladsb1.books.repository.BookRepository;
import com.cladsb1.books.repository.CategoryRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;
import java.util.Optional;

@Service
public class BookService {
 private final BookRepository bookRepository;
 private final CategoryRepository categoryRepository;

 public BookService(BookRepository bookRepository,
 CategoryRepository categoryRepository) {
 this.bookRepository = bookRepository;
 this.categoryRepository = categoryRepository;
 }

 public List<BookResponseDTO> findAll() {
 return bookRepository.findAll()
 .stream()
 .map(this::toDTO)
 .toList();
 }

 public Optional<BookResponseDTO> findById(Long id) {
 return bookRepository.findById(id)
 .map(this::toDTO);
 }

 @Transactional
 public BookResponseDTO create(BookRequestDTO dto) {
 Category category = resolveCategory(dto.category());
 Book saved = bookRepository.save(new Book(dto.title(),
 dto.author(), category));
 return toDTO(saved);
 }

 @Transactional
 public Optional<BookResponseDTO> update(Long id, BookRequestDTO dto) {
 return bookRepository.findById(id)
 .map(existing -> {
 Category category = resolveCategory(dto.category());
 existing.setTitle(dto.title());
 existing.setAuthor(dto.author());
 existing.setCategory(category);
 return bookRepository.save(existing);
 })
 .map(this::toDTO);
 }

 @Transactional
 public boolean delete(Long id) {
 if (!bookRepository.existsById(id)) return false;
 bookRepository.deleteById(id);
 return true;
 }

 /**
 * Resuelve la categoría:
 * - si existe -> la devuelve
 * - si no existe -> la crea
 */
 private Category resolveCategory(String categoryName) {
 return categoryRepository.findByNameIgnoreCase(categoryName)
 .orElseGet(() -> categoryRepository.save(new Category(categoryName)));
 }

 private BookResponseDTO toDTO(Book book) {
 return new BookResponseDTO(
 book.getId(),
 book.getTitle(),
 book.getAuthor(),
 book.getCategory().getName()
 );
 }
}
```

🟦 Bloque 5A — BookController limpio (CON lambdas)

📁 **controller/BookController.java**

```
package com.cladsb1.books.controller;

import com.cladsb1.books.dto.request.BookRequestDTO;
import com.cladsb1.books.dto.response.BookResponseDTO;
import com.cladsb1.books.service.BookService;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.net.URI;
import java.util.List;

@RestController
@RequestMapping("/api/books")
public class BookController {
 private final BookService bookService;

 public BookController(BookService bookService) {
 this.bookService = bookService;
 }

 @GetMapping
 public ResponseEntity<List<BookResponseDTO>> getAll() {
 return ResponseEntity.ok(bookService.findAll());
 }

@GetMapping("/{id}")
public ResponseEntity<BookResponseDTO> getById(@PathVariable Long id) {
return bookService.findById(id)
.map(ResponseEntity::ok)
.orElse(ResponseEntity.notFound().build());
}

@PostMapping
public ResponseEntity<BookResponseDTO> create(@Valid @RequestBody BookRequestDTO dto) {
BookResponseDTO created = bookService.create(dto);
URI location = URI.create("/api/books/" + created.id());
return ResponseEntity.created(location).body(created);
}

@PutMapping("/{id}")
public ResponseEntity<BookResponseDTO> update(@PathVariable Long id,
@Valid @RequestBody BookRequestDTO dto) {
return bookService.update(id, dto)
.map(ResponseEntity::ok)
.orElse(ResponseEntity.notFound().build());
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) {
return bookService.delete(id)
? ResponseEntity.noContent().build()
: ResponseEntity.notFound().build();
}
}
```

✅ B) VERSIÓN SIN STREAMS / SIN LAMBDAS

🟦 Bloque 4B — BookService (SIN streams/lambdas)

📁 **service/BookService.java**

```
package com.cladsb1.books.service;

import com.cladsb1.books.dto.request.BookRequestDTO;
import com.cladsb1.books.dto.response.BookResponseDTO;
import com.cladsb1.books.entity.Book;
import com.cladsb1.books.entity.Category;
import com.cladsb1.books.repository.BookRepository;
import com.cladsb1.books.repository.CategoryRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

@Service
public class BookService {
 private final BookRepository bookRepository;
 private final CategoryRepository categoryRepository;

 public BookService(BookRepository bookRepository,
 CategoryRepository categoryRepository) {
 this.bookRepository = bookRepository;
 this.categoryRepository = categoryRepository;
 }

 public List<BookResponseDTO> findAll() {
 List<Book> books = bookRepository.findAll();
 List<BookResponseDTO> result = new ArrayList<>();
 for (Book b : books) {
 result.add(toDTO(b));
 }
 return result;
 }

 public Optional<BookResponseDTO> findById(Long id) {
 Optional<Book> opt = bookRepository.findById(id);
 if (opt.isEmpty()) return Optional.empty();
 return Optional.of(toDTO(opt.get()));
 }

 @Transactional
 public BookResponseDTO create(BookRequestDTO dto) {
 Category category = resolveCategory(dto.category());
 Book book = new Book(dto.title(), dto.author(), category);
 Book saved = bookRepository.save(book);
 return toDTO(saved);
 }

 @Transactional
 public Optional<BookResponseDTO> update(Long id, BookRequestDTO dto) {
 Optional<Book> opt = bookRepository.findById(id);
 if (opt.isEmpty()) return Optional.empty();

 Book existing = opt.get();
 Category category = resolveCategory(dto.category());
 existing.setTitle(dto.title());
 existing.setAuthor(dto.author());
 existing.setCategory(category);
 Book saved = bookRepository.save(existing);
 return Optional.of(toDTO(saved));
 }

 @Transactional
 public boolean delete(Long id) {
 boolean exists = bookRepository.existsById(id);
 if (!exists) return false;
 bookRepository.deleteById(id);
 return true;
 }

 private Category resolveCategory(String categoryName) {
 Optional<Category> opt = categoryRepository.findByNameIgnoreCase(categoryName);
 if (opt.isPresent()) {
 return opt.get();
 }
 return categoryRepository.save(new Category(categoryName));
 }

 private BookResponseDTO toDTO(Book book) {
 return new BookResponseDTO(
 book.getId(),
 book.getTitle(),
 book.getAuthor(),
 book.getCategory().getName()
 );
 }
}
```

🟦 Bloque 5B — BookController limpio (SIN lambdas)

📁 **controller/BookController.java**

```
package com.cladsb1.books.controller;

import com.cladsb1.books.dto.request.BookRequestDTO;
import com.cladsb1.books.dto.response.BookResponseDTO;
import com.cladsb1.books.service.BookService;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.net.URI;
import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/books")
public class BookController {
 private final BookService bookService;

 public BookController(BookService bookService) {
 this.bookService = bookService;
 }

 @GetMapping
 public ResponseEntity<List<BookResponseDTO>> getAll() {
 List<BookResponseDTO> result = bookService.findAll();
 return ResponseEntity.ok(result);
 }

 @GetMapping("/{id}")
public ResponseEntity<BookResponseDTO> getById(@PathVariable Long id) {
Optional<BookResponseDTO> opt = bookService.findById(id);
if (opt.isEmpty()) {
return ResponseEntity.notFound().build();
}
return ResponseEntity.ok(opt.get());
}

@PostMapping
public ResponseEntity<BookResponseDTO> create(@Valid @RequestBody BookRequestDTO dto) {
BookResponseDTO created = bookService.create(dto);
URI location = URI.create("/api/books/" + created.id());
return ResponseEntity.created(location).body(created);
}

@PutMapping("/{id}")
public ResponseEntity<BookResponseDTO> update(@PathVariable Long id,
@Valid @RequestBody BookRequestDTO dto) {
Optional<BookResponseDTO> opt = bookService.update(id, dto);
 if (opt.isEmpty()) {
 return ResponseEntity.notFound().build();
 }
 return ResponseEntity.ok(opt.get());
 }

 @DeleteMapping("/{id}")
 public ResponseEntity<Void> delete(@PathVariable Long id) {
 boolean deleted = bookService.delete(id);
 if (!deleted) {
 return ResponseEntity.notFound().build();
 }
 return ResponseEntity.noContent().build();
 }
}
```

🟦 Bloque 6 — Tabla: operación → status

GET all	200	Controller
GET by id existe	200	Controller
GET by id no existe	404	Controller
POST válido	201	Controller
POST inválido	400	Validation + Handler
PUT existe	200	Controller
PUT no existe	404	Controller
DELETE existe	204	Controller
DELETE no existe	404	Controller

🟦 Bloque 7 — Errores típicos

❌ Meter categoryRepository en el controller
"porque es rápido"

❌ Hacer @Transactional en controller
(mala práctica)

❌ Repetir mapping en 5 métodos distintos
(se centraliza en toDTO / MapStruct luego)

❌ Poner reglas de negocio en repository
(no es su rol)

🟦 Bloque 8 — Cierre

💡 Mensajes finales:

"El controller no piensa."

"El service hace la operación atómica."

"HTTP en controller, negocio en service."